

## Task 2 – Planning & Designing a Procedural Programming Application

### 1. Requirements Gathering

Before planning and designing the application, the client was interviewed and the following requirements were gathered:

#### Interview questions

- What will this application do and who will be using it?
- Is there a preference for a programming paradigm and a programming language for this application?
- How many employees does the application need to support?
- What information needs to be stored in the application?
- What calculations does the application need to do?
- What information does the application need to run those calculations?
- What will the application output to the terminal? Is there a format to follow?
- What should the application use for output? A separate UI or the terminal?
- What menu screens are needed and how should the navigation work?

The answers to these questions shaped the business and technical requirements below.

#### Business Requirements:

- Application will be used by payroll department. Only payroll team can use the application.
- Application will do the actions below:
  - display employees and detailed employee profiles,
  - add new employees to the system,
  - remove employees from the system,
  - generate a payslip for each employee,
  - edit payroll details for each employee.
- Application should support up to 50 employees.
- The information stored in the application will be:
  - Employee full name
  - Employee department and role info
  - Employee salary and PPS number
- The information needed for the application to run will be:
  - Irish PAYE, PRSI and USC rates and standard tax credits for tax calculations
  - Employee ID and salary for displaying employee profiles and payslips
- Application should be able to calculate the following for each employee:
  - Annual PAYE, PRSI, USC and total deductions
  - Annual and monthly deductions and net pay
  - Programmatically give the pay period and payment date for each payslip
- Payslip format should look similar to [this example](#).

## Technical Requirements:

- Application should be CLI-based, written in Python and follow a procedural programming approach.
- Application should validate these user inputs and show explanatory error messages to guide the user:
  - Employee ID, PPS number and salary data when adding a new employee
  - User input when user is making choices while navigating the system
- Navigation should allow the user to jump from one menu to another and cancel an action if desired.
- Payslip should have a formatted view and be properly displayed in a (small) laptop screen since most users use a laptop rather than large desktop monitors nowadays.

## 2. Inputs, Processing, Outputs

All inputs are of `str` type as Python's `input()` function returns a string. When the inputs are used in payroll calculations, they are converted to `Decimal` for more precise financial arithmetic.

Employee records are stored as a dictionary in a .json file (`employees.json`). The dictionary keys, when needed, are converted to a `List` so indexed access and pagination is possible.

| Feature                  | Function                             | Input                                                             | Processing                                                                                | Output                                                                  |
|--------------------------|--------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Display main menu        | <code>display_main_menu</code>       | User input: 1 to 4                                                | Takes and validates user input                                                            | Routes to submenus                                                      |
| Display employee list    | <code>display_employees()</code>     | User input: 1 to 4                                                | Loads data from <code>employees.json</code>                                               | Paginated employee list and menu options                                |
| Display employee menu    | <code>display_employee_menu()</code> | User input: <code>Employee ID</code> or <code>c</code> for cancel | Takes and validates user input                                                            | Prompt to get user input and routes to <code>display_profile(id)</code> |
| Display employee profile | <code>display_profile(id)</code>     | <code>Employee ID</code> and User input: 1 to 4                   | Looks up and validates ID in employees dictionary                                         | Detailed employee profile and employee menu options                     |
| Display payslip          | <code>display_payslip(id)</code>     | <code>Employee ID</code> and <code>salary</code>                  | Looks up salary data in employees dictionary and calls functions for payslip calculations | Employee payslip                                                        |

| Feature               | Function                         | Input                                                                                                                             | Processing                                                     | Output                                                                                                    |
|-----------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Add new employee      | <code>add_new_employee()</code>  | User input:<br><code>name</code> ,<br><code>department</code> ,<br><code>role</code> , <code>ppsn</code> ,<br><code>salary</code> | Validates user input and updates <code>employees.json</code>   | Records new data in <code>employees.json</code>                                                           |
| Remove employee       | <code>remove_employee(id)</code> | Employee <code>id</code> and user input: <code>y</code> or <code>n</code>                                                         | Prompt to confirm or cancel deleting the employee record       | Either delete record from <code>employees.json</code> or routes to <code>display_profile(id)</code>       |
| Edit employee details | <code>edit_details(id)</code>    | Employee <code>id</code> and user input: <code>c</code> or <code>salary</code>                                                    | Validates user input and writes to <code>employees.json</code> | Routes to <code>display_profile(id)</code> with or without updating record in <code>employees.json</code> |
| Display help menu     | <code>display_help()</code>      | User input:<br><code>any key</code>                                                                                               | Displays help menu                                             | Help content                                                                                              |

### 3. Employee Records Data Structure

Key: Employee ID (e.g. ID12) Value: [name, department, role, salary, ppsn]

For example: "ID1": ["Alice Murphy", "Engineering", "Software Developer", "34000", "1234567AB"]

### 4. Input validations

| Function                          | Input            | Validation Rule                                                                                            |
|-----------------------------------|------------------|------------------------------------------------------------------------------------------------------------|
| <code>is_id(str)</code>           | Employee ID      | Must start with ID                                                                                         |
| <code>is_id_present(str)</code>   | Employee ID      | ID must exist in the system                                                                                |
| <code>is_valid_answer(str)</code> | 'y' or 'n'       | Must be 'y' or 'n' or 'Y' or 'N'                                                                           |
| <code>is_valid_salary(str)</code> | Salary           | Must be numeric only and 5-6 digits                                                                        |
| <code>is_valid_length(str)</code> | Employee details | Must be max 20 characters                                                                                  |
| <code>is_valid_ppsn(str)</code>   | PPS Number       | Must be 7 numerals followed by one or two letters. It must be unique and cannot be duplicate in the system |
| <code>is_valid_option(str)</code> | 1,2,3,4          | Must be 1,2,3 or 4                                                                                         |

### Application Flowchart

The flowchart below illustrates the navigation in the system

